C++ Programming Problem Set

Module: Classes & Objects (Encapsulation & Memory Layout)

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Classes, Objects & Scope Resolution (::)

Integrates: Encapsulation, References, Arrays, Strings & Functions

Instructions & Pedagogical Guidelines

This problem set focuses on Object-Oriented Programming foundations in C++. You will write programs establishing strict encapsulation with `private` data members and `public` member functions, apply scope resolution membership labels (`ClassName::function`), analyze RAM object footprints vs shared code segment methods, and implement caller-object vs argument-object arithmetic.

Level 1: Encapsulation, Scope Resolution & Memory Footprints

Problem 1: Encapsulated Coordinate Point & Distance Calculator

Classes & Objects | Scope Resolution (::) | Encapsulation

Design a class `Point2D` with private data members `int x` and `int y`. Declare member functions inside the class body and define them *outside* using the Scope Resolution Operator (`void Point2D::setCoordinates(int, int)` and `void Point2D::display()`). Implement a public method `double distanceToOrigin()`. Demonstrate in `main()` that accessing `p1.x` directly from `main()` triggers a compilation access error.

Private Data Hiding | Scope Resolution (::) | Public Interface

Call Site Example	Output & Diagnostic Behavior
`Point2D p1; p1.setCoordinates(3, 4);`	`Coordinates: (3, 4) | Distance to Origin: 5.00`
`std::cout << p1.x;`	`ERROR: 'x' is a private member of 'Point2D'!`

Problem 2: Object RAM Footprint & Shared Code Segment Inspector

Memory Architecture | sizeof(Object) | Implicit Inlining

Define a class `MemoryInspector` containing `int id` (4 bytes), `double balance` (8 bytes), and 3 member functions. Instantiate 3 separate object instances. Print `sizeof(MemoryInspector)` and the physical RAM address of each instance to verify that objects allocate space exclusively for data members on the Stack, while all instances share a single copy of member functions in the Code Segment.

Stack Object Allocation | Shared Code Segment | Implicit Inline Methods

Instance Name	Memory Analysis Output
`obj1, obj2, obj3`	`sizeof(Object): 16 Bytes (Data only + Padding)` `obj1 Addr: 0x7ffd10 | obj2 Addr: 0x7ffd20 (+16 Bytes offset)`

Problem 3: Inline Member Methods vs External Membership Labels

Inline Methods | Membership Label | Binary Size

Create a class `MathUtility`. Define a tiny getter method `int getVal()` directly *inside* the class body (implicitly inlined by the compiler). Declare a larger processing function `void processArray(int arr[], int size)` inside the class and define it *outside* using `void MathUtility::processArray(...)`. Explain in comments how inlining small methods boosts execution speed while external definitions prevent binary code bloat.

Implicit Inlining | External Definition | Binary Optimization

Method Placement	Compiler Handling
`Inside Class Body`	`Implicitly Inlined (In-place code expansion)`
`Outside Class Body (::)`	`Compiled as standard function in Code Segment`

Level 2: Caller vs Argument Objects, References & Array Processing

Problem 4: Caller vs Argument Object Addition (Complex Numbers)

Caller Object | Argument Object | Return Object

Implement a class `Complex` with private integers `real` and `imag`. Write a method `Complex add(const Complex &c)`. In `c3 = c1.add(c2);`, identify that `c1` is the Caller Object (accessed directly via `real/imag`), `c2` is the Passed Argument (accessed via `c.real/c.imag`), and a temporary `Complex temp` object is constructed and returned to `c3`.

```
// Caller vs Argument Interaction inside Complex::add: Complex add(const Complex &c) const { Complex temp; temp.real = real + c.real; // 'real' = c1 (caller), 'c.real' = c2 (argument) temp.imag = imag + c.imag; return temp; }
```

Call Site	Execution Result Summary
`c1(3, 4), c2(5, 6) -> c3 = c1.add(c2)`	`c3 State -> Real: 8 | Imaginary: 10`

Problem 5: In-Place Time Accumulator (Pass-by-Const-Reference)

Const Reference | Time Normalization | Member Access

Design a class `Time` with private members `hrs`, `mins`, and `secs`. Implement methods:
1. `void setTime(int h, int m, int s)` with normalization logic (e.g., 75s -> 1m 15s).
2. `Time addTime(const Time &t2) const` returning a new normalized `Time` object.
Show why passing `const Time &t2` prevents making expensive object copies on the call stack frame while enforcing read-only safety.

Const Reference Parameter | Normalization Algorithm | Read-Only Safety

Inputs Passed	Calculated Result
`T1 = 01:45:50 | T2 = 02:20:30`	`T3 = T1.addTime(T2) -> Result: 04:06:20 (Normalized)`

Problem 6: Array of Objects Student Merit Roster

Array of Objects | Object Search | Encapsulation

Create a class `Student` with private members `rollNo`, `name` (std::string), and `marks`. Instantiate an array of 5 `Student` objects in `main()`: `Student roster[5];`. Write a external helper function `void findTopScorer(const Student arr[], int size)` that iterates through the object array, invoking `getMarks()` to locate and print the top student's record.

Array of Objects | Getter Interfaces | Object Iteration

Array of Objects	Roster Query Result
`roster[0..4] populated with scores`	`Top Scorer Found -> Roll #102: Alice (95.5 Marks)`

Level 3: Dynamic Heap Objects, Struct vs Class & Banking Capstone

Problem 7: Dynamic Heap Object Allocation & Arrow Operator

Dynamic Memory | new / delete | Arrow Operator (->)

Demonstrate dynamic instantiation of objects on the Heap. Define a class `BankAccount` with private balance data. In `main()`, dynamically instantiate an object using `BankAccount *accPtr = new BankAccount();`. Call member methods via the arrow operator (`accPtr->deposit(500.0)`). Compute interest, print updated balances, and clean up heap storage using `delete accPtr;`.

```
// Dynamic Object Lifetime Management on Heap: BankAccount *accPtr = new BankAccount(); accPtr->setData(1001, 1500.0); accPtr->display(); delete accPtr; // Free dynamic Heap memory!
```

Heap Allocation	Output Summary
`new BankAccount() on Heap`	`Account #1001 | Balance: $1,500.00 | Deallocated cleanly via delete`

Problem 8: Technical Equivalence Matrix (class vs struct)

class vs struct | Default Access Visibility | Access Specifiers

Write a comparative program proving that in C++, `class` and `struct` have identical technical capabilities except for default visibility. Define `struct StructPoint { int x; };` and `class ClassPoint { int x; };` without access labels. Show that `StructPoint s; s.x = 10;` compiles because struct defaults to `public`, whereas `ClassPoint c; c.x = 10;` fails because class defaults to `private`.

struct Default Public | class Default Private | Technical Equivalence

Construct	Default Member Access Visibility
`struct StructPoint`	`PUBLIC by default (Accessible directly in main)`
`class ClassPoint`	`PRIVATE by default (Gated; requires explicit public: label)`

Problem 9: Cumulative Capstone: Automated Banking Ledger Engine

Comprehensive Integration | Classes & Objects | System Engine

Build a complete banking system engine. Design a class `BankAccount` with private members `accNo` (int), `holderName` (std::string), and `balance` (double). Implement member methods defined outside using `::`:
1. `void setDetails(int acc, std::string name, double initialBalance)`
2. `bool withdraw(double amount)` — Returns false if funds are insufficient.
3. `void transfer(BankAccount &targetAcc, double amount)` — Caller object transfers funds to target object.

Process a system of 3 accounts in `main()`, executing deposits, withdrawals, and inter-account transfers.

Encapsulated Banking Engine | Inter-Object Transfer | Scope Resolution Methods | Complete Module Capstone

Transaction Operations	Ledger State Output
`Acc #101 ($1000) transfers $300 to Acc #102 ($500)`	`Acc #101 Balance: $700.00` `Acc #102 Balance: $800.00 (Transfer Successful)`